

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»  
(МОСКОВСКИЙ ПОЛИТЕХ)

# КУРСОВОЙ ПРОЕКТ

по курсу  
«Разработка веб-приложений»

## ТЕМА

«Разработка веб-приложения для управления проектами и  
отслеживания задач»

**Выполнил:** Набиуллин Артур Фанилевич

**Группа:** 241-326

**Проверил:** Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»  
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ  
заведующая кафедрой  
«Инфокогнитивные технологии»

\_\_\_\_\_ / Е. А. Пухова /

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

**ЗАДАНИЕ**

**на выполнение курсовой работы (проекта)**

Набиуллину Артуру Фанилевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления проектами и отслеживания задач»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

| Разрабатываемый вопрос                                                               | Объем,<br>% | Срок       | Примечание |
|--------------------------------------------------------------------------------------|-------------|------------|------------|
| <b>Раздел 1. Анализ предметной области</b>                                           |             |            |            |
| Задача 1.1. Обзор существующих программных продуктов по теме работы                  | 10          | 05.04.2026 |            |
| Задача 1.2. Анализ программных инструментов разработки веб-приложений                | 7           | 11.04.2026 |            |
| Задача 1.3. Формулировка цели и задач работы                                         | 8           | 18.04.2026 |            |
| <b>Раздел 2. Проектирование веб-приложения</b>                                       |             |            |            |
| Задача 2.1. Анализ целевой аудитории                                                 | 5           | 25.04.2026 |            |
| Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования) | 7           | 02.05.2026 |            |

| Разрабатываемый вопрос                                                                           | Объем,<br>% | Срок       | Примечание |
|--------------------------------------------------------------------------------------------------|-------------|------------|------------|
| Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)        | 8           | 09.05.2026 |            |
| Задача 2.4. Разработка макетов страниц (Wireframe)                                               | 5           | 16.05.2026 |            |
| <b>Раздел 3. Разработка веб-приложения</b>                                                       |             |            |            |
| Задача 3.1. Реализация подсистем аутентификации и авторизации пользователей                      | 8           | 21.05.2026 |            |
| Задача 3.2. Реализация системы распределения ролей и проверки прав доступа к функциям приложения | 7           | 26.05.2026 |            |
| Задача 3.3. Реализация CRUD-интерфейсов для управления проектами и задачами                      | 7           | 30.05.2026 |            |
| Задача 3.4. Разработка функциональности загрузки и скачивания файлов, прикрепленных к задачам    | 12          | 03.06.2026 |            |
| Задача 3.5. Реализация модуля агрегирования данных для расчёта и вывода статистики по проектам   | 6           | 06.06.2026 |            |
| <b>Раздел 4. Оформление итогов работы</b>                                                        |             |            |            |
| Задача 4.1. Создание Git-репозитория с кодом проекта                                             | 3           | 09.06.2026 |            |
| Задача 4.2. Деплой приложения на хостинг                                                         | 4           | 11.06.2026 |            |
| Задача 4.3. Оформление отчёта о проделанной работе                                               | 3           | 13.06.2026 |            |

Руководитель курсовой работы (проекта):  
старший преподаватель кафедры «Инфокогнитивные технологии»

«\_\_\_\_» \_\_\_\_\_ 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

« 31 » марта 2026 г.

Дата сдачи выполненной работы

« 13 » июня 2026 г.

Задание принял к исполнению

« 31 » марта 2026 г.

(подпись)

А. Ф. Набиуллин

## СОДЕРЖАНИЕ

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ</b>                                               | <b>5</b>  |
| <b>1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ</b>                            | <b>6</b>  |
| 1.1 Обзор существующих программных продуктов по теме работы   | 6         |
| 1.2 Анализ программных инструментов разработки веб-приложений | 6         |
| 1.3 Формулировка цели и задач работы . . . . .                | 8         |
| <b>2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ</b>                        | <b>10</b> |
| 2.1 Анализ целевой аудитории . . . . .                        | 10        |
| 2.2 Описание функциональности приложения . . . . .            | 11        |
| 2.3 Проектирование модели данных . . . . .                    | 13        |
| 2.4 Разработка макетов страниц . . . . .                      | 15        |
| <b>3 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ</b>                            | <b>18</b> |
| 3.1 Реализация подсистемы аутентификации и проверки ролей . . | 18        |
| 3.2 Разработка CRUD-интерфейсов и работы с файлами . . . . .  | 20        |
| 3.3 Реализация модуля агрегирования данных . . . . .          | 21        |
| <b>ЗАКЛЮЧЕНИЕ</b>                                             | <b>22</b> |

## ВВЕДЕНИЕ

**Актуальность темы.** В современных условиях цифровизации бизнеса и повсеместного перехода на удаленный или гибридный формат работы, эффективное управление проектами становится критически важным фактором успеха любой организации. Использование традиционных методов учета задач (электронные таблицы, мессенджеры) приводит к потере информации, срыву сроков и снижению производительности команды. Разработка специализированных веб-приложений для трекинга задач позволяет централизовать рабочие процессы, автоматизировать контроль поручений и обеспечить прозрачность работы на всех этапах жизненного цикла проекта.

**Степень разработанности проблемы.** На рынке существует множество систем управления проектами (Jira, Trello, Asana и др.). Однако большинство корпоративных решений обладают избыточным функционалом, сложны в освоении и имеют высокую стоимость лицензий, что делает их нерентабельными для малого бизнеса, студенческих команд или индивидуального использования. В связи с этим, создание легковесного, интуитивно понятного веб-приложения с базовым, но достаточным набором функций (CRUD-операции, ролевая модель, прикрепление файлов) остается актуальной практической задачей.

**Объект исследования** — процессы управления проектами, распределения задач и контроля их выполнения в рабочих группах.

**Предмет исследования** — программные средства, алгоритмы и методы веб-разработки, применяемые для создания систем отслеживания задач.

**Целью данной работы** является проектирование и реализация клиент-серверного веб-приложения, предназначенного для эффективного управления проектами и отслеживания статусов выполнения задач с учетом ролевой модели доступа.

## АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

### 1.1 Обзор существующих программных продуктов по теме работы

На современном рынке программного обеспечения для управления проектами представлено множество решений. Для выявления недостатков существующих систем и формирования требований к разрабатываемому веб-приложению проведен структурированный анализ популярных платформ: Jira, Trello и Asana.

Сравнение проводилось по ключевым критериям, критически важным для небольших рабочих групп: наличие полнофункциональной бесплатной версии, простота освоения, поддержка ролевой модели доступа (RBAC), наличие встроенной статистики и возможность прикрепления файлов. Результаты сравнительного анализа представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ систем управления проектами

| Критерий оценки              | Jira | Trello     | Asana      | Наш проект |
|------------------------------|------|------------|------------|------------|
| Бесплатный полный функционал | –    | –          | –          | +          |
| Простота освоения            | –    | +          | +          | +          |
| Ролевая модель доступа       | +    | – (платно) | – (платно) | +          |
| Графическая статистика       | +    | –          | +          | +          |
| Прикрепление файлов          | +    | +          | +          | +          |

Анализ показал, что мощные корпоративные системы (Jira) обладают необходимым функционалом, но перегружены и сложны для малого бизнеса. Более простые решения (Trello, Asana) искусственно ограничивают функционал в бесплатных версиях, вынуждая покупать подписку ради доступа к статистике или ролевым моделям. Это подтверждает целесообразность разработки собственного легковесного веб-приложения.

### 1.2 Анализ программных инструментов разработки веб-приложений

Для реализации клиент-серверной архитектуры проекта необходимо выбрать оптимальный стек технологий. Сравнение производилось по трем ключевым направлениям: серверная часть (Backend), база данных и клиентская

часть (Frontend).

**1. Выбор Backend-фреймворка.** Рассматривались популярные решения на языке Python: *Django*, *FastAPI* и *Flask*.

- *Django* отличается монолитной архитектурой «всё включено», имеет встроенную ORM и панель администратора. Однако его избыточность негативно сказывается на скорости разработки небольших проектов.
- *FastAPI* выделяется асинхронной архитектурой и высокой производительностью, идеально подходит для построения микросервисов (API). Недостаток — сложность интеграции с классическим серверным рендерингом страниц.
- *Flask* [1] предоставляет минималистичное ядро с возможностью точечного подключения нужных библиотек (высокая модульность).

**Вывод:** Фреймворк Flask выбран как оптимальный баланс между легковесностью и функциональностью, позволяющий реализовать приложение без избыточного кода.

**2. Выбор системы управления базами данных (СУБД).** Сравнение проводилось между *SQLite*, *MySQL* и *PostgreSQL*.

- *SQLite* хранит данные в одном локальном файле, не требует настройки сервера, но блокирует базу целиком при записи, что исключает её использование в многопользовательской среде.
- *MySQL* обладает высокой скоростью чтения, однако исторически имеет менее строгую проверку типов данных и сложную систему изменения таблиц.
- *PostgreSQL* является мощной объектно-реляционной СУБД, поддерживающей механизм многоверсионности (MVCC) для параллельной работы без блокировок, строгую ссылочную целостность и сложные аналитические запросы.

**Вывод:** Для надежного хранения данных пользователей и работы с агрегацией статистики в production-среде выбрана СУБД PostgreSQL, взаимодействующая с кодом через ORM-библиотеку SQLAlchemy [2].

**3. Выбор Frontend-технологий.** Сравнивались подходы Single Page Application (SPA) и Server-Side Rendering (SSR).

- Использование SPA-фреймворков (*React* или *Vue.js*) позволяет создать высокоинтерактивный интерфейс без перезагрузки страниц, но требует настройки отдельного сервера сборки (Node.js/Webpack) и усложняет маршрутизацию запросов (CORS).
- Подход SSR на базе шаблонизатора *Jinja2* позволяет генерировать HTML-страницы непосредственно на Python-сервере. В связке с CSS-библиотекой *Bootstrap 5* это обеспечивает быструю отрисовку и адаптивность под мобильные устройства «из коробки».

**Вывод:** Для интерфейса Task Tracker'а динамики связки Jinja2 + Bootstrap 5 более чем достаточно, что исключает необходимость в избыточной SPA-архитектуре.

### 1.3 Формулировка цели и задач работы

На основе проведенного анализа предметной области (п. 1.1) и выбора технологического стека (п. 1.2), главная **цель работы** заключается в создании веб-приложения для управления проектами и отслеживания задач на базе фреймворка Flask и СУБД PostgreSQL.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и существующих решений;
2. Спроектировать пользовательские сценарии (Use Cases) и проанализировать целевую аудиторию;
3. Спроектировать логическую и физическую модели реляционной базы данных;
4. Разработать макеты пользовательского интерфейса (Wireframes);
5. Реализовать подсистему аутентификации пользователей и настроить систему распределения ролей;



6. Разработать программные модули (CRUD-интерфейсы) для управления сущностями «Проект», «Задача» и «Пользователь»;
7. Реализовать функционал загрузки и скачивания файлов, прикрепленных к задачам;
8. Разработать модуль агрегирования данных для вывода аналитической статистики по проектам;
9. Развернуть готовое веб-приложение на удаленном сервере (хостинге).

## ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

### 2.1 Анализ целевой аудитории

Основной целевой аудиторией разрабатываемого веб-приложения являются малые рабочие группы, стартапы, IT-команды на начальном этапе развития и студенческие проектные коллективы. Как правило, численность таких команд составляет от 3 до 20 человек.

В ходе анализа были выявлены основные проблемы (боли) данной аудитории при организации совместной работы традиционными методами:

- Потеря информации и артефактов (файлов) при обсуждении рабочих процессов в неспециализированных мессенджерах;
- Неочевидность статуса выполнения проекта в целом (отсутствие понимания, какой сотрудник занят конкретной задачей в данный момент);
- Высокий порог входа и перегруженность интерфейса в крупных корпоративных системах, что требует излишнего времени на обучение новых сотрудников.

Для удовлетворения этих потребностей и обеспечения безопасности системы была спроектирована ролевая модель доступа (RBAC). Исходя из паттернов поведения пользователей, выделены две ключевые роли:

- **Администратор (Менеджер проекта):** выступает в роли руководителя или координатора. Обладает полными правами в системе. Занимается созданием новых проектов, управлением списком задач и осуществляет глобальный контроль за ходом выполнения работ через аналитические дашборды.
- **Пользователь (Исполнитель):** выступает в роли рядового участника команды. Имеет ограниченный доступ: видит только те проекты, в которые был приглашен. Основная задача данной роли — брать задачи в работу, изменять их статус по мере выполнения и прикреплять файлы с результатами работы.

## 2.2 Описание функциональности приложения

Разрабатываемое веб-приложение предоставляет пользователям набор инструментов для совместной работы и эффективного управления проектами. Основная функциональность системы включает в себя:

- Регистрацию, аутентификацию и управление сессиями пользователей;
- Разграничение прав доступа к функциям в зависимости от роли пользователя (администратор или исполнитель);
- Выполнение полного цикла операций (создание, чтение, обновление, удаление) для управления проектами и задачами;
- Загрузку и скачивание файлов, прикрепляемых к конкретным задачам в качестве отчетов;
- Автоматическое агрегирование данных для расчета процента готовности проектов и вывода визуальной статистики.

Взаимодействие пользователей с системой строится вокруг следующего базового сценария: Администратор авторизуется в системе, создает новый проект и формирует список задач, назначая исполнителей. Пользователь, заходя в систему, видит назначенную ему задачу, переводит её в статус «В процессе», а по завершении работы загружает файл с результатом и меняет статус на «Готово».

Визуальное представление доступных функций и разграничения прав доступа между ролями продемонстрировано на диаграмме вариантов использования (Use Case) на рисунке 2.1.

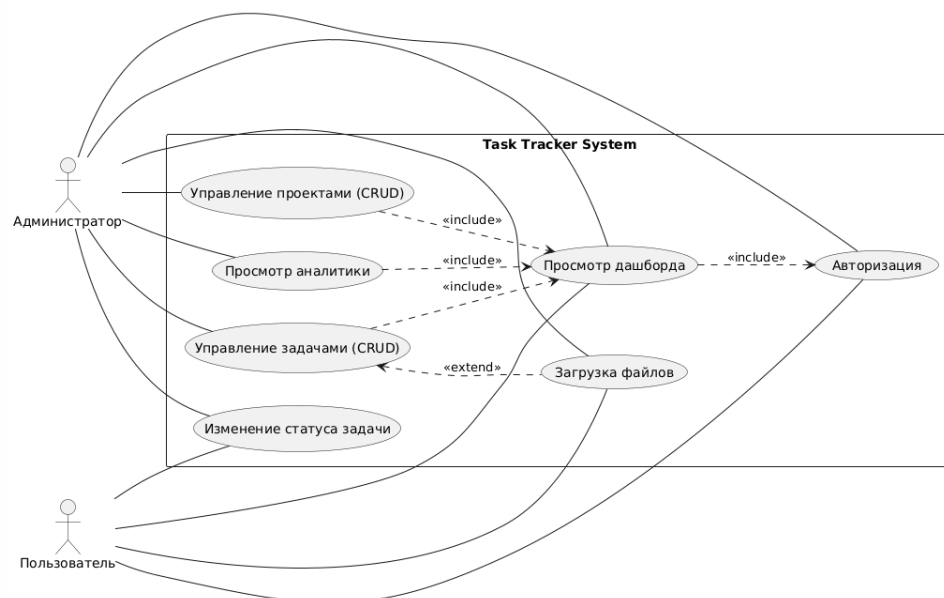


Рисунок 2.1 – Диаграмма вариантов использования системы

## 2.3 Проектирование модели данных

Проектирование базы данных веб-приложения осуществлялось в два этапа: разработка логической и физической схем. На рисунке 2.2 представлена ER-диаграмма, иллюстрирующая связи между основными объектами системы.

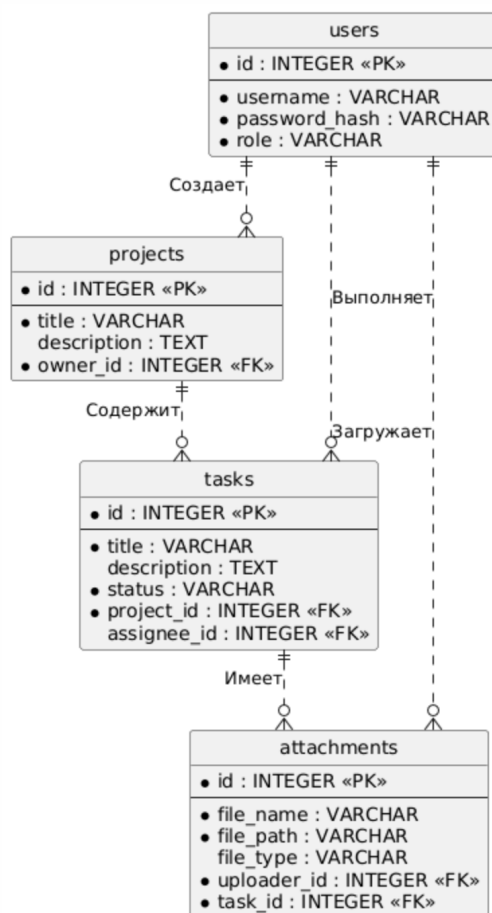


Рисунок 2.2 – ER-диаграмма базы данных

**Логическая схема базы данных** включает четыре основные сущности: users (Пользователи), projects (Проекты), tasks (Задачи) и attachments (Вложения).

**Физическая схема базы данных** описывает реализацию таблиц в СУБД с указанием типов данных и ограничений. Структуры разработанных таблиц представлены в таблицах 2.1 — 2.4.

Таблица 2.1 – Структура таблицы пользователей (users)

| Поле          | Тип данных   | Ограничения и описание |
|---------------|--------------|------------------------|
| id            | INTEGER      | PRIMARY KEY            |
| username      | VARCHAR(50)  | UNIQUE, NOT NULL       |
| password_hash | VARCHAR(128) | NOT NULL (хэш пароля)  |
| role          | VARCHAR(20)  | DEFAULT 'user'         |

Таблица 2.2 – Структура таблицы проектов (projects)

| Поле        | Тип данных   | Ограничения и описание |
|-------------|--------------|------------------------|
| id          | INTEGER      | PRIMARY KEY            |
| title       | VARCHAR(100) | NOT NULL               |
| description | TEXT         | NULL                   |
| owner_id    | INTEGER      | FOREIGN KEY → users.id |

Таблица 2.3 – Структура таблицы задач (tasks)

| Поле        | Тип данных   | Ограничения и описание           |
|-------------|--------------|----------------------------------|
| id          | INTEGER      | PRIMARY KEY                      |
| title       | VARCHAR(100) | NOT NULL                         |
| description | TEXT         | NULL (Детальное описание задачи) |
| status      | VARCHAR(20)  | DEFAULT 'pending'                |
| project_id  | INTEGER      | FOREIGN KEY → projects.id        |
| assignee_id | INTEGER      | FOREIGN KEY → users.id           |

Таблица 2.4 – Структура таблицы вложений (attachments)

| Поле        | Тип данных   | Ограничения и описание      |
|-------------|--------------|-----------------------------|
| id          | INTEGER      | PRIMARY KEY                 |
| file_name   | VARCHAR(255) | NOT NULL (Оригинальное имя) |
| file_path   | VARCHAR(255) | NOT NULL (Путь на сервере)  |
| file_type   | VARCHAR(20)  | NULL (Тип: pdf, docx и др.) |
| uploader_id | INTEGER      | FOREIGN KEY → users.id      |
| task_id     | INTEGER      | FOREIGN KEY → tasks.id      |

## 2.4 Разработка макетов страниц

Для обеспечения высокого уровня пользовательского опыта (UX) интерфейс приложения спроектирован в минималистичном стиле. Для разгрузки основной таблицы проектов функции просмотра деталей задачи и загрузки файлов были вынесены в отдельные модальные окна.

На рисунке 2.3 представлен макет панели управления (дашборда), а на рисунке 2.4 — макет детальной страницы проекта с таблицей задач.

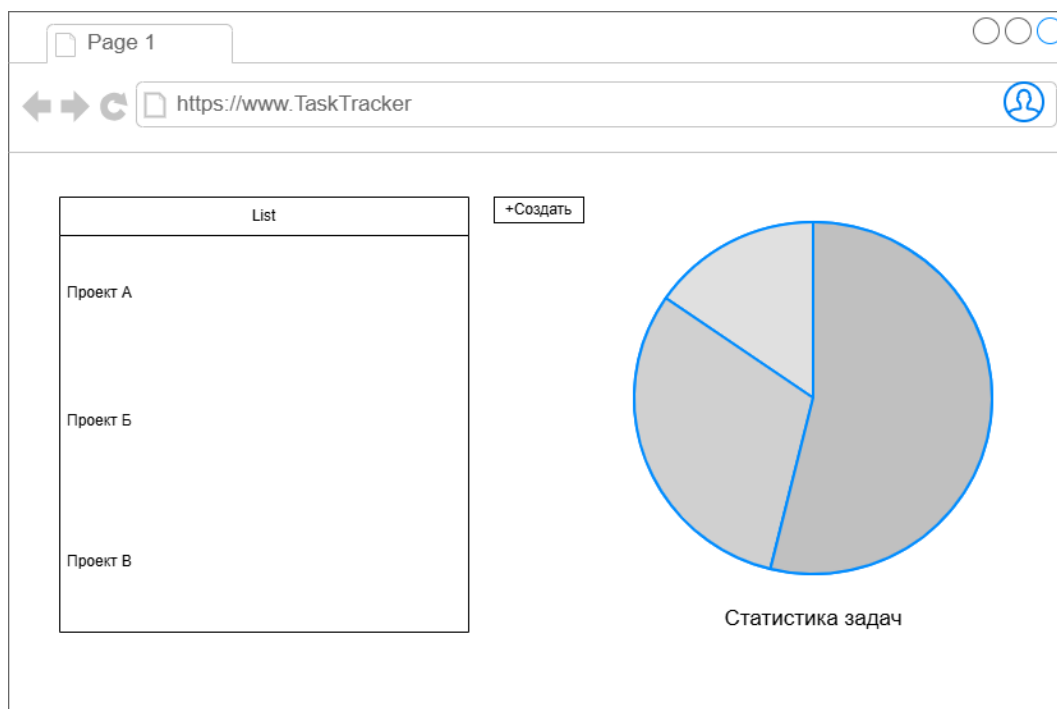


Рисунок 2.3 – Макет главной панели управления (Дашборд)

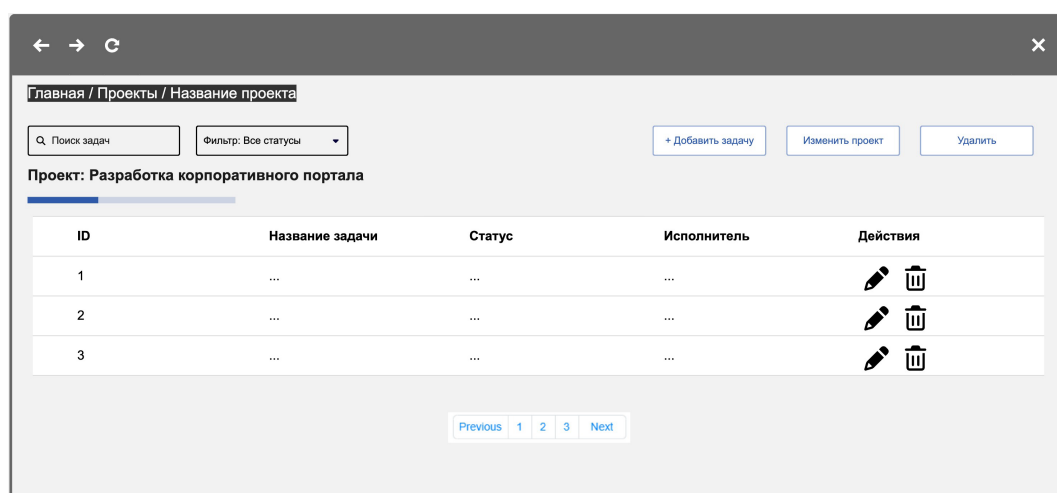


Рисунок 2.4 – Макет страницы проекта со списком задач

Для демонстрации структуры дополнительных окон ниже представлены фактические экранные формы реализованного веб-приложения, утвержденные в рамках проектирования интерфейса. На рисунках 2.5 — 2.6 представлены окна управления проектом.

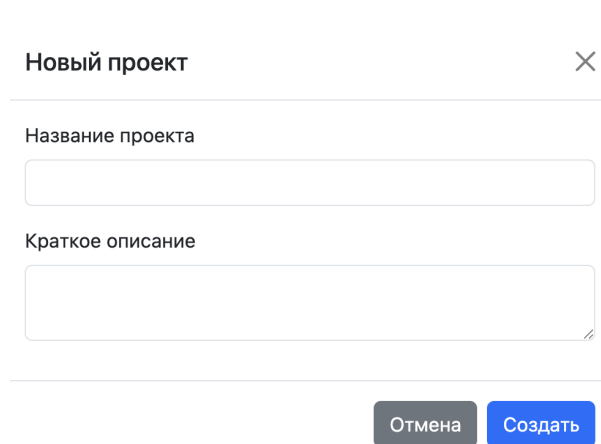


Рисунок 2.5 – Окно создания проекта

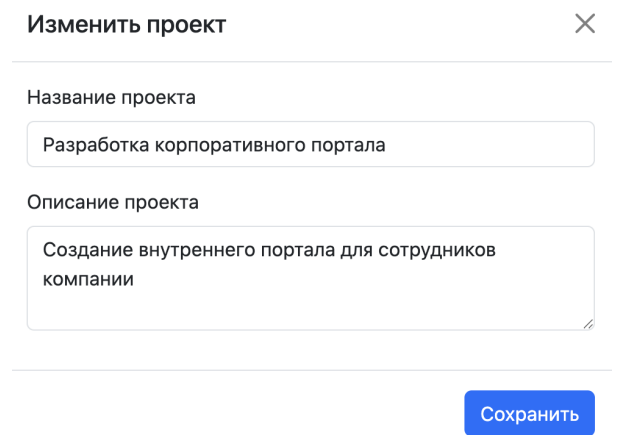


Рисунок 2.6 – Окно редактирования проекта

На рисунках 2.7 — 2.8 представлены окна создания и редактирования задач. На рисунке 2.9 представлено окно просмотра, содержащее полное описание задачи и список прикрепленных файлов.

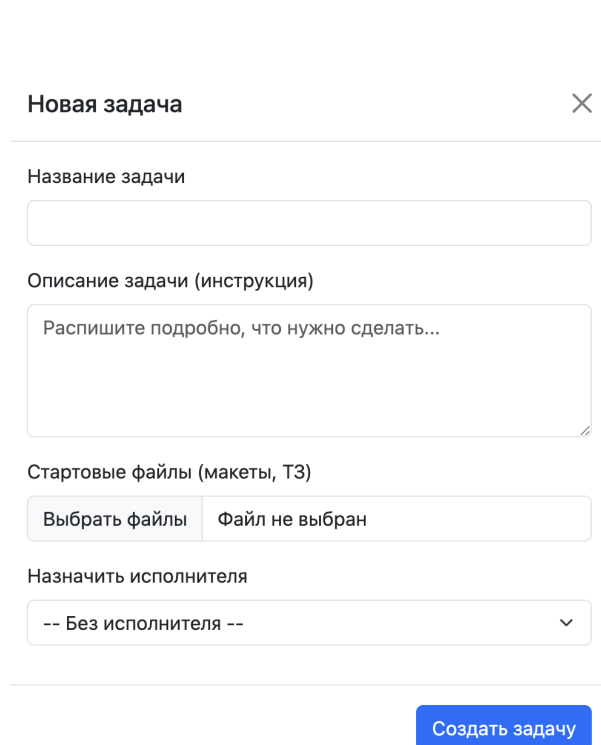


Рисунок 2.7 – Окно создания задачи

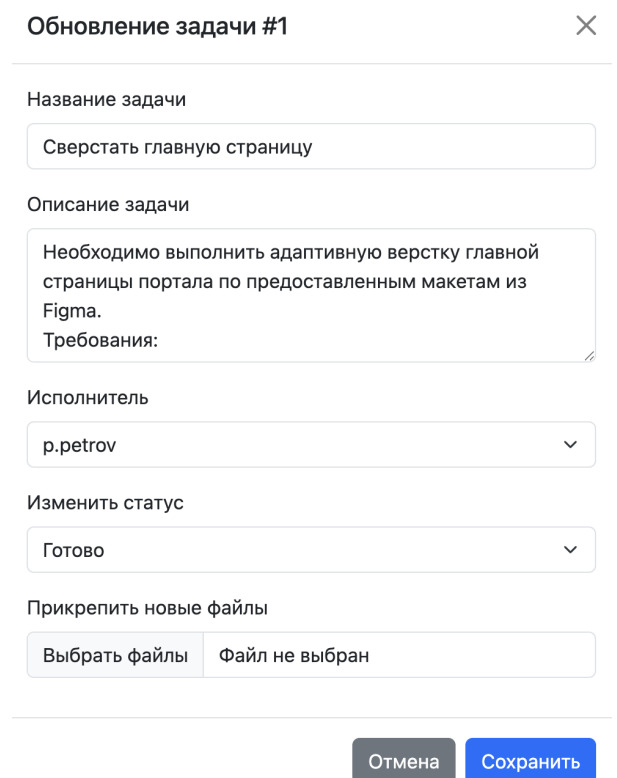


Рисунок 2.8 – Окно редактирования задачи



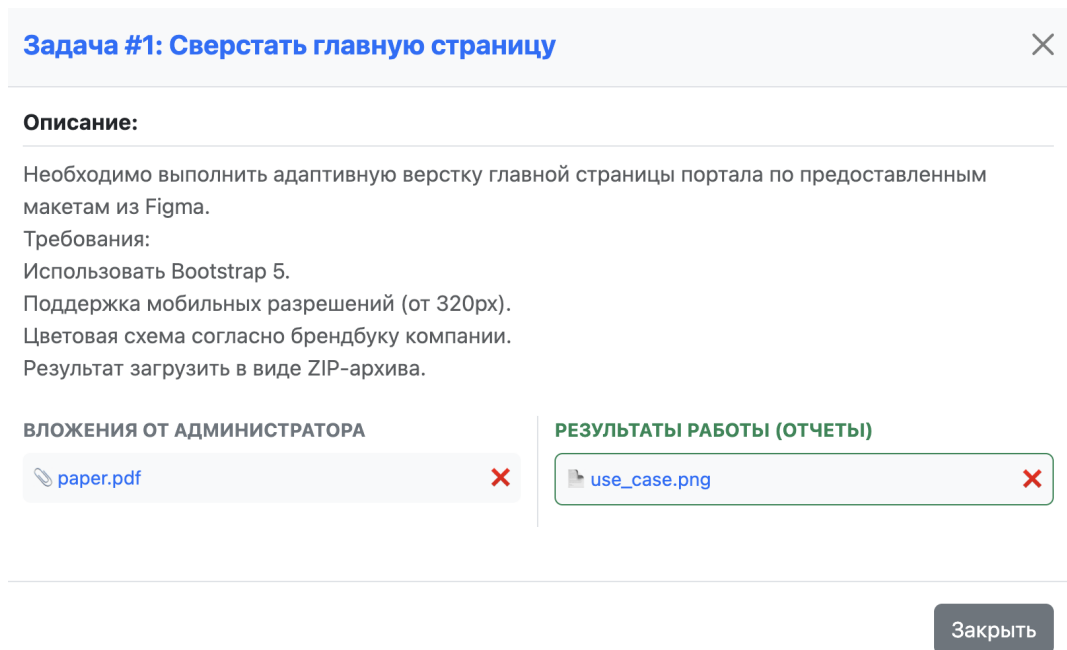


Рисунок 2.9 – Окно просмотра описания и файлов задачи

## РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

### 3.1 Реализация подсистемы аутентификации и проверки ролей

Основой безопасности разрабатываемого приложения является подсистема аутентификации, реализованная с помощью библиотеки Flask-Login. Хранение паролей в базе данных осуществляется в зашифрованном виде с использованием криптографических хэш-функций из модуля `werkzeug.security` (функции `generate_password_hash` и `check_password_hash`).

Для разграничения прав доступа была внедрена ролевая модель (RBAC). При выполнении критически важных действий на стороне сервера происходит проверка атрибута `role` текущего пользователя.

Листинг 1 — Фрагмент кода проверки прав доступа при создании проекта

```
1 @app.route('/create_project', methods=['POST'])
2 @login_required
3 def create_project():
4     if current_user.role != 'admin':
5         flash('У вас нет прав для создания проектов.', 'danger')
6         return redirect(url_for('index'))
7
8     title = request.form.get('title')
9     description = request.form.get('description')
10
11     if title:
12         new_project = Project(title=title, description=description,
13                               owner_id=current_user.id)
14         db.session.add(new_project)
15         db.session.commit()
16     return redirect(url_for('index'))
```

На рисунках 3.1 и 3.2 представлены интерфейсы страниц регистрации и входа в систему.

The screenshot shows the registration page of a web application. At the top, a dark header bar contains the text "TaskTracker" on the left and "Вход Регистрация" on the right. The main content area has a light gray background. In the center, there is a white box titled "Регистрация". Inside this box, there are two input fields: "Логин" (Login) and "Пароль" (Password). Below these fields is a blue button with the text "Зарегистрироваться" (Register).

Рисунок 3.1 – Страница регистрации

The screenshot shows the login page of the same web application. The header bar is identical to the previous page. The main content area has a light gray background. In the center, there is a white box titled "Вход в систему" (Login to the system). Inside this box, there are two input fields: "Логин" (Login) and "Пароль" (Password). Below these fields is a checkbox labeled "Запомнить меня" (Remember me). At the bottom of the box is a green button with the text "Войти" (Login).

Рисунок 3.2 – Страница авторизации

## 3.2 Разработка CRUD-интерфейсов и работы с файлами

Управление сущностями системы осуществляется через реализованные CRUD-интерфейсы. Взаимодействие с реляционной базой данных PostgreSQL инкапсулировано через объектно-реляционное отображение (ORM) SQLAlchemy.

Особое внимание уделено безопасной работе с файлами и возможности множественного прикрепления документов к одной задаче. Для защиты файловой системы сервера используется утилита `secure_filename`, а для предотвращения коллизий реализована генерация уникальных префиксов на базе библиотеки `uuid`. Кроме того, внедрен механизм автоматического физического удаления файлов с жесткого диска при удалении проекта или задачи.

На рисунке 3.3 продемонстрирован интерфейс управления задачами конкретного проекта, включающий таблицу задач, элементы фильтрации и навигации.

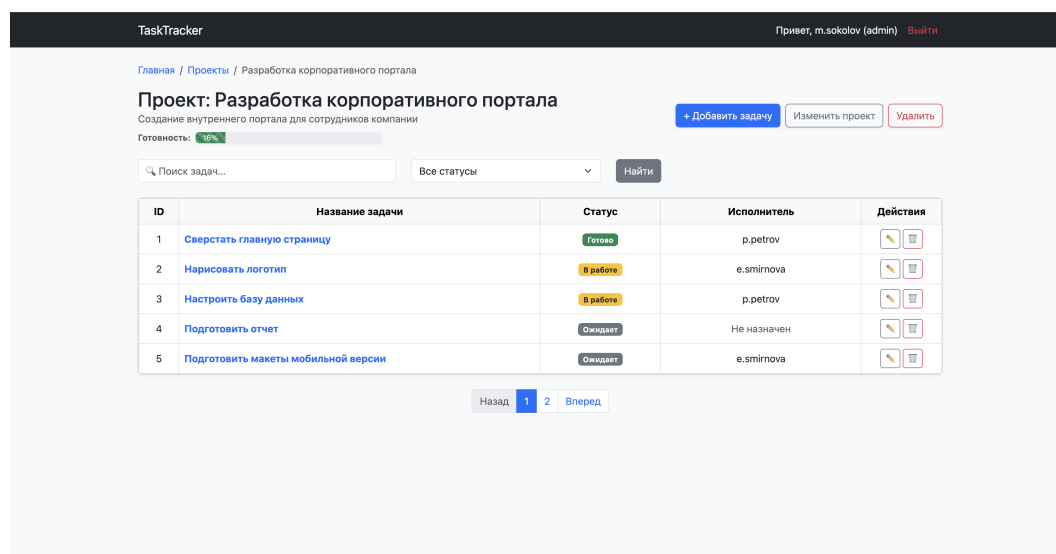


Рисунок 3.3 – Интерфейс управления задачами проекта

### 3.3 Реализация модуля агрегирования данных

Для обеспечения администратора системы наглядной аналитикой был разработан модуль агрегирования данных. На серверной стороне с помощью агрегатных функций SQLAlchemy (метод `.count()`) выполняется подсчет задач, сгруппированных по их текущему статусу.

Полученные статистические данные передаются в HTML-шаблоны, где визуализируются в двух форматах: в виде динамической шкалы готовности (Progress Bar) внутри каждого проекта и в виде интерактивной круговой диаграммы на базе библиотеки `Chart.js` на главной панели управления. Результат работы модуля представлен на рисунке 3.4.

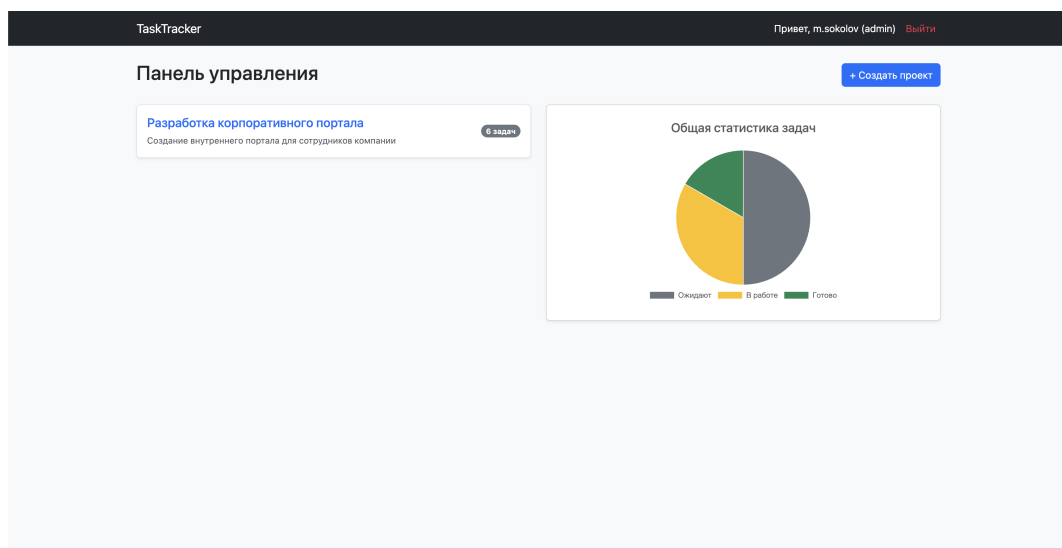


Рисунок 3.4 – Главная панель управления (Дашборд) с круговой диаграммой

Листинг 2 — Фрагмент кода агрегации данных в базе

```
1 # Агрегирование данных для круговой диаграммы
2 total_tasks = Task.query.count()
3 pending_tasks = Task.query.filter_by(status='pending').count()
4 in_progress_tasks = Task.query.filter_by(status='in_progress').count()
5 done_tasks = Task.query.filter_by(status='done').count()
6
7 stats = {
8     'total': total_tasks,
9     'pending': pending_tasks,
10    'in_progress': in_progress_tasks,
11    'done': done_tasks
12 }
```

## ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно достигнута поставленная цель — спроектировано и разработано полноценное клиент-серверное веб-приложение для управления проектами и отслеживания задач.

В результате выполнения работы были решены следующие задачи:

1. Проведен анализ предметной области, который подтвердил актуальность создания легковесных и интуитивно понятных систем управления задачами (Task Trackers) для малых рабочих групп.
2. Спроектирована архитектура приложения с использованием микрофреймворка Flask (Python) и объектно-реляционного отображения SQLAlchemy, обеспечивающая высокую производительность и безопасность взаимодействия с базой данных PostgreSQL.
3. Разработаны пользовательские интерфейсы (Wireframes), которые впоследствии были успешно перенесены в интерактивные HTML-шаблоны с применением CSS-фреймворка Bootstrap 5.
4. Внедрена надежная подсистема аутентификации и реализована ролевая модель (RBAC), разделяющая функционал системы между «Администраторами» и обычными «Пользователями».
5. Спроектирована логическая и физическая структура базы данных, состоящая из четырех сущностей (Пользователи, Проекты, Задачи, Вложения). Реализован функционал управления данными (на базе CRUD-операций), при этом полный жизненный цикл (включая редактирование и удаление) реализован для основной бизнес-сущности — «Задачи».
6. Интегрирован функционал безопасной загрузки, хранения и скачивания файлов-отчетов, прикрепляемых к конкретным задачам.
7. Разработан аналитический модуль агрегирования данных, который в реальном времени визуализирует статистику выполнения задач с использованием графической библиотеки Chart.js.

Разработанная система полностью удовлетворяет техническим требованиям, отличается высокой скоростью отклика, защищенностью (шифрование

паролей, защита от SQL-инъекций) и адаптивным дизайном, и может служить надежным инструментом для автоматизации рабочих процессов в малых и средних командах.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [3], определяющие структуру и правила оформления научно-исследовательских работ.

Исходный код разработанного веб-приложения размещен в открытом репозитории на платформе GitHub и доступен по ссылке: <https://github.com/sqeezeeee/web-app-coursework>.

## СПИСОК ЛИТЕРАТУРЫ

- [1] Flask Documentation (3.0.x) [Электронный ресурс] [Text]. — URL: <https://flask.palletsprojects.com/> (дата обращения: 01.04.2026).
- [2] SQLAlchemy 2.0 Documentation[Электронный ресурс] [Text]. — URL: <https://docs.sqlalchemy.org/> (дата обращения: 01.04.2026).
- [3] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.